

SCIO Clone — Build Plan

Author: Lam Nhat Hao · **Date:** 2 Oct 2026 · **Audience:** the engineering team that will build this, and the leadership who will fund it.

How to read this. Section 1 says what SCIO is in my words. Section 2 is every scoping decision, with the reason, and the assumptions I am making. Section 3 is the system design, including capacity numbers and what happens when each part fails. Sections 4–5 are the build order and the estimate, with a range and a confidence per milestone. Section 6 is the one thing that surprised me and what it changed. Section 7 is what I would cut first if the number is too big.

1. What SCIO actually is

SCIO is a multi-tenant control plane for screens. A customer pairs a device (a stick, a TV, a laptop) by typing a short pair code into the portal, and from then on the portal decides *what* that screen shows, *when*, in *what layout*, and *how the device itself behaves*. Everything else is in service of that loop.

After using the product and reading the full help center (416 articles, which I scraped for Part 1), the product decomposes into six subsystems. The article counts are a rough proxy for where customer complexity actually lives.

Subsystem	What it covers	Help-center weight
Fleet	Pairing, screens, folders, tags, device settings, remote reboot/screenshot, online status, mass provisioning, operational schedules (power on/off)	~60 articles
Content	Assets (upload, transcode, storage), Websites/URLs, Designer templates, Canva	~30
Composition	Playlists (nested, per-item duration, playback rules), Split-screen zones, Schedules with default content	~20
Apps	100+ integrations: YouTube, Google/Microsoft apps, Social, Weather, News, Stock, Calendar, Dashboards, OptiSync data mapping	~150
Organisation	Users, roles, folder-level permissions, SSO/SAML, billing and plan gating, teams	~30
Players	12+ platforms: Android/Fire TV, Windows, macOS, Linux, Raspberry Pi, ChromeOS, BrightSign, LG webOS, Samsung Tizen, Roku, Apple TV, web browser	~70

Two things that are easy to miss from the outside. First, the player is not a dumb renderer: it caches everything locally, keeps playing when the network drops, buffers proof-of-play events offline and syncs them later. Second, the business model is feature gating by plan (Free → Standard → Pro → Pro Plus → Engage → Enterprise). Almost every "Advanced" article opens with which plan unlocks it. The clone has to model entitlements from day one or retrofitting them touches every endpoint.

2. Decisions the brief left open

I am deciding these, and saying why, so the live discussion can challenge the decision rather than the ambiguity.

Goal. A v1 that three design-partner customers (multi-location retail or restaurant, 50 to 300 screens each) run in production by month 8, and that can be sold to similar customers at month 10. Not a feature-complete replica: parity with 100+ apps is a years-long catalogue effort and not where the engineering risk is.

Team: 6 engineers plus the three roles plans usually forget, 10 months.

- 1 tech lead (architecture, API, reviews; codes about half the time)
- 2 backend (API, media pipeline, sync protocol)
- 2 frontend (portal, player UI)
- 1 Android/player engineer (native shell, video surface, caching, device commands, update channel)
- 1 QA engineer who owns the device lab and the end-to-end suite, from M1, not later
- 0.5 DevOps (infrastructure as code, CI/CD, observability) from M0
- 0.5 product designer from M0 to M3, when the portal and player UI are being shaped
- A product owner on the business side, not counted in engineering capacity

Why 6 engineers: the product has four independently deep tracks (API, portal, player, media pipeline). Fewer than 5 means tracks serialise. More than 7 and the tech lead stops coding, and the pairing/sync protocol, which is the hardest part, gets designed by committee. Why QA from M1: the device matrix is the main risk in Section 5, and nobody finds autostart or scaling bugs on six different sticks without a person whose job it is.

Platforms: web portal, web player, Android player. Android covers Fire TV, Android sticks, tablets, and the OptiStick-class hardware, which is the bulk of real deployments. The web player runs in any browser and gives us a Windows/macOS/Linux player via a thin Electron wrapper. The Android player is a **hybrid**: a WebView for layout and apps, a native ExoPlayer surface for video zones, because WebView video stutters on cheap sticks and that is the first thing a customer notices. Samsung Tizen, LG webOS, BrightSign, Roku, Apple TV are each a separate codebase with vendor certification; they are out of v1.

Stack: TypeScript end-to-end. React + Next.js portal, NestJS API, PostgreSQL with row-level security per tenant, Redis, S3 + CDN with signed URLs for media, MQTT (EMQX) over WSS on port 443 for device push, FFmpeg workers for transcoding. One language means the player, portal, and API share the content schema and the schedule-resolution engine, which is the single most bug-prone piece of logic in a signage product. Choosing Go or Java for the API would be defensible but would fork that logic.

In scope for v1

Area	Included
Fleet	Pair by code, per-device identity and tokens, screens, folders, tags, heartbeat/online status, remote reboot, screenshot, device settings, device-level operational schedule (black screen on/off), player self-update with staged rollout, fleet alerts (screen offline > 10 minutes)
Content	Image/video/PDF upload, transcoding to a fixed rendition ladder, thumbnails, Website/URL asset, Canva/Slides embed as the "design" path
Composition	Playlists incl. nested and per-item duration, split-screen zones with 4 preset layouts, weekly recurring schedules with date ranges and default content, resolved by the same engine on server and player
Apps	An app framework (sandboxed iframe + config schema) plus 5 reference apps that need no OAuth: YouTube, Website, Weather, RSS, Clock
Organisation	Orgs, teams, users, invite, default and custom roles, folder-level permissions, audit log, SAML SSO, Stripe billing with plan gating and quotas (screens per team, storage)

Area	Included
Players	Web player with offline cache, hybrid Android player, Electron desktop wrapper
API	Public REST with API keys and webhooks

Deferred to v1.1 (first quarter after launch), not cut

Deferred	Why not v1
Designer (canvas editor)	10 to 14 weeks of frontend that absorbs unlimited polish; design partners can use Canva or Slides embeds, which many customers already do
Proof-of-play	10 000 screens emitting an event every 10 s is ~86 million rows a day; it needs its own pipeline (player-side rollups, ClickHouse or partitioned tables), not a Postgres table bolted onto v1
GraphQL API	REST covers the integrations design partners asked for; GraphQL adds a second surface to secure and document
OAuth apps (Google Slides, Microsoft, social)	Per-tenant OAuth consent and token refresh is 3 to 4 weeks of infrastructure; it ships as one piece in v1.1 and unlocks a dozen apps at once

Deliberately out of v1 and v1.1, and why

Cut	Why
The other 95+ apps	Catalogue work, each is 1–3 days once the framework and OAuth exist; no architectural risk
OptiSync data mapping / repeaters	A product in itself (spreadsheet → template binding engine)
Engage / Kiosk touch, Kiosk Designer Pro	Different interaction model, different plan, different buyer
HDMI-CEC / RS-232 hardware control	Needs owned hardware and per-TV quirks; device-level on/off covers most of the use case
Tizen, webOS, BrightSign, Roku, Apple TV players	Vendor certification cycles; add one per quarter after launch
AI Camera, Sensors/IoT, OptiSound, Emergency alerts	Niche, each needs partners or licensing
Teams Rooms / Zoom Rooms / Meet hardware discovery	Depends on third-party admin APIs

Assumptions and open questions for leadership

- We sell software only; customers bring Android sticks or Fire TV. If we sell our own stick, add a hardware provisioning track and 8 to 10 weeks.
- Single region, no data-residency requirement in v1. A second region is an infrastructure change, not a code change, if tenancy stays strict.
- Three design partners are signed by the end of M1 and give us weekly feedback and real screens. Without them the exit criteria in Section 4 are untestable.
- Target 10 000 screens across all tenants at launch, 1 000 per tenant, growing to 100 000 within two years. Section 3.6 shows where the design needs to change on the way there.

3. System design

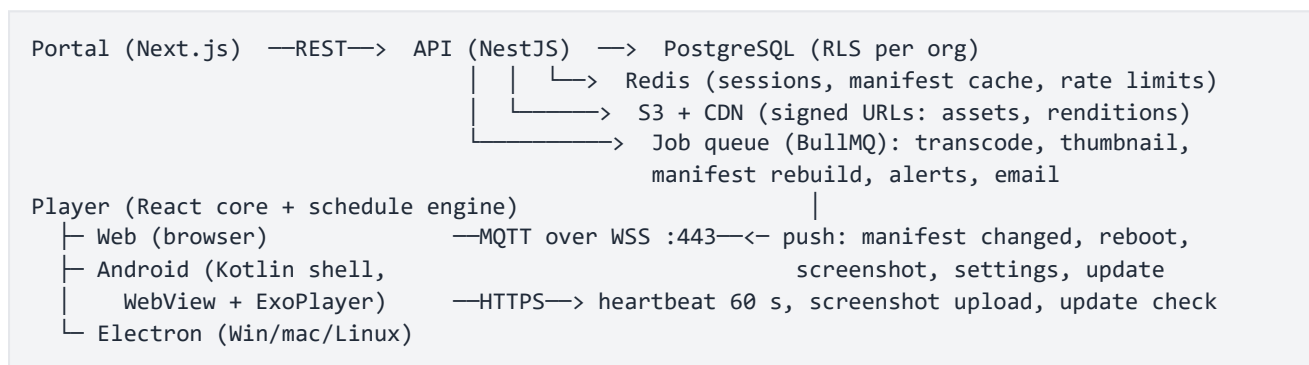
3.1 The core loop: rules down, events up, play from cache

The design centre is a **per-screen manifest**: a versioned JSON document holding the screen's schedule rules, playlists, zones, and the asset set, with every asset as a signed CDN URL plus content hash. The player downloads the manifest and every asset it references into local storage, then runs the **same TypeScript schedule engine the server uses** to decide what to show at each moment. A change in the portal bumps the manifest version and pushes a short MQTT message; the player fetches the new manifest and only the assets it does not already have.

Why this shape:

- The player never needs the network to keep playing, for hours or for weeks. Resolving on the player, not only on the server, is what makes offline duration unbounded. The server runs the same engine for preview and validation.
- The engine is one pure package with one test suite; portal preview, server validation and player playback cannot drift apart.
- The player is thin and the same core runs on web, in the Android WebView, and in Electron. The Android shell adds only what the web cannot do: autostart, a native video surface, local storage without quota, self-update.

3.2 Components



3.3 The write path: what happens when someone clicks Save

1. The portal calls the API; the API validates, writes inside one transaction, and appends an `audit_log` row.
2. The transaction enqueues a **manifest rebuild job** for every screen affected. A playlist edit touches every screen that plays it, directly or through a nested playlist, so the API resolves the dependency graph (asset → playlist → layout/schedule → screen) from indexed join tables, not by scanning.
3. Workers rebuild each manifest, store it in Postgres and in Redis, and publish `{screen_id, version}` on that screen's MQTT topic. Rebuilds are idempotent and coalesced: ten edits in a minute produce one rebuild per screen, with the latest version winning.
4. The player receives the push, fetches the manifest over HTTPS (never over MQTT, which carries only notifications), diffs the asset set by content hash, downloads what is missing from the CDN, and swaps to the new version atomically at the next item boundary.

A customer with 300 screens who edits a company-wide playlist triggers 300 rebuilds; at about 20 ms each on one worker that is 6 s, and workers scale horizontally.

3.4 Device identity and security

- Pair codes are 6 characters, expire in 15 minutes, and are rate-limited per IP and per account.
- Pairing issues a per-device token bound to a device record; tokens rotate every 30 days and are revoked on unpair. Every heartbeat, manifest fetch and MQTT subscription presents it.
- MQTT topics are per device and enforced by broker ACLs, so a compromised stick can only read its own channel.
- Asset URLs are signed and expire in 24 hours; the player refreshes the manifest before they lapse. Customer content never sits on a public URL.
- Every portal mutation writes an audit log row (who, what, before/after). Multi-location operators ask for this in the first month.

3.5 Player self-update

A fleet is only as good as its ability to update itself. Players report their version in every heartbeat. Releases are staged by tag (internal, design partners, 10 %, everyone) and can be held or rolled back per tag. The manifest carries a schema version, and the API serves older schema versions for players that have not yet updated, so an update never bricks a screen. Consumer devices use the Play Store; sticks use a self-hosted APK channel.

3.6 Capacity numbers and failure modes

Sized for the launch target (10 000 screens) with the path to 100 000 noted.

Load	At 10 000 screens	At 100 000	What changes
Heartbeats (60 s)	~170 req/s, one small row update each	~1 700 req/s	Write heartbeats to Redis, flush "last seen" to Postgres every 5 min; online/offline alerts read Redis
MQTT connections	10 000 persistent WSS, idle	100 000	EMQX clusters; one node handles ~100 000 idle connections, so this is a two-node change
Manifest rebuilds	Peaks of a few hundred per minute after a bulk edit	Tens of thousands per minute	Already queued and coalesced; add worker pods
Asset egress	Bulk of bytes; a 50 MB video to 300 screens is 15 GB through the CDN, zero through the API	Same shape	CDN handles it; origin sees one fetch per asset
Postgres	A few hundred writes/s, single primary with a read replica	Thousands/s	Partition screen and manifest by org; still one primary

What happens when each part fails:

Failure	Screens	Portal	Recovery
API down	Keep playing from cache indefinitely; heartbeats fail silently and retry with backoff	Down	Stateless API pods behind a load balancer; redeploy

Failure	Screens	Portal	Recovery
MQTT broker down	Keep playing; the player polls the manifest version over HTTPS every 5 min as a fallback, so changes are delayed, not lost	Edits save normally	Broker restarts; players reconnect with jittered backoff so 10 000 reconnects do not arrive in the same second
Postgres primary down	Keep playing	Read-only until failover	Managed failover to the replica in about 60 s; the job queue replays rebuilds
CDN or S3 outage	Keep playing cached assets; new assets wait	Uploads fail with a clear error	Nothing to do on our side; the manifest swap is atomic, so a half-downloaded asset set is never shown
One bad player release	Only the rollout tag is affected	Unaffected	Roll back the tag; players self-update to the previous build
A tenant floods the API	Rate-limited per API key and per org	Unaffected tenants	Limits enforced in Redis before the request reaches Postgres

The invariant behind all of this: **the player must never depend on the backend to keep showing what it was last told to show**. Every decision above follows from it.

3.7 Domain model (the tables that matter)

```
org → team, user, membership(team, role), role(capabilities), folder(kind: screen|asset|playlist,
permission), screen(pair_code, device_token, device_info, player_version, folder, tags, settings,
current_manifest_version), asset(type, renditions, folder, tags),
playlist(items[asset|playlist|app], folder), layout(zones[]), schedule(rules[], default_content),
app_instance(app_type, config), assignment(screen → asset|playlist|layout|schedule),
manifest(screen, version, schema_version, json), audit_log, plan, entitlement, quota.
```

Every row carries `org_id`; PostgreSQL row-level security enforces tenancy below the application layer. This is cheap to add on day one and nearly impossible to add later.

3.8 Quality, delivery, SLOs and cost

Testing. The schedule engine gets property-based tests (random rule sets across timezones and DST boundaries must always resolve to exactly one item per zone). The API gets contract tests per endpoint and an RLS test that runs every query as a second tenant and expects zero rows. End-to-end tests run nightly on the six lab devices against staging; a device simulator (the player core in headless mode) runs the 10 000-screen load test in M5 and a 500-screen smoke test on every merge.

Delivery. Trunk-based development, feature flags per tenant for anything customer-visible, player releases through the rollout tags in 3.5, a two-week change freeze before launch. Every merge deploys to staging; production deploys are daily and boring.

SLOs we commit to from M4. API availability 99.9 % monthly. A content change reaches p95 of its screens within 30 s. Player online rate, measured by heartbeats, at least 99.5 % per tenant, excluding screens the customer has powered off. Offline alert within 10 minutes of the last heartbeat. These are also the numbers the status page shows.

Retention. Heartbeats 30 days, audit log 1 year, screenshots 7 days, deleted assets recoverable for 30 days.

Infrastructure cost at 10 000 screens, order of magnitude. CDN egress dominates: at 2 GB of new content per screen per month that is 20 TB, roughly 1 500 to 2 000 USD. API, workers, Postgres with a replica, Redis and a two-node EMQX cluster are another 2 000 to 3 000 USD. Call it 5 000 USD a month, or 0.50 USD per screen, before storage and support tooling. The assumption to challenge is the 2 GB per screen; a video-heavy customer can triple it, which is why storage and egress are quota-gated by plan.

4. Build order and why

Each milestone ends with something a design partner can run. Order is driven by **risk first, then value**: the sync protocol, the media pipeline and the Android device matrix are where unknowns live, so they come before any feature breadth. Exit criteria are measurable on purpose; a milestone that cannot be measured cannot be declared done.

#	Milestone	Weeks	Exit criterion (measured, not felt)	Why here
M0	Foundations	1–4	Monorepo, CI/CD, staging, infra as code, auth + orgs + invites, Postgres schema with RLS, S3/CDN with signed URLs, logs/metrics/alerts. Device lab stocked with 6 reference devices.	Everything after this is parallel; nothing can start without tenancy, deploy and devices
M1	A screen shows a picture	4–10	Pair in < 10 s; upload image/video, transcoded; a change reaches 100 screens within 30 s; a player unplugged from the network for 24 h keeps playing and resumes cleanly; device tokens rotate and revoke.	Proves the manifest/engine/cache/push loop end to end. If this is wrong, nothing else matters
M2	Playlists, schedules, Android	10–18	Nested playlists, weekly schedules with default content, folders, tags; hybrid Android player plays 1080p video loops without a dropped frame on all 6 lab devices; self-update rolls a build to the "design partners" tag and back. First design partner live on 20 screens.	The first thing a customer buys. Android here because it is the real-world device and exposes caching, video and autostart issues early
M3	Layouts, apps, desktop	18–24	Split-screen with 4 layouts; Website asset; app framework + 5 apps; Electron wrapper; p95 portal page load < 1.5 s on a 1 000-screen tenant.	Composition breadth on top of a stable player; apps are a day each once the framework exists
M4	Fleet and organisation	24–34	Teams, roles, folder permissions, audit log; remote reboot/screenshot/settings; operational schedule; fleet offline alerts; mass provisioning; REST API + keys + webhooks. All three design partners live, 300+ screens total.	Needed for multi-location buyers. After M3 so the permission model is tested against real resource types with real users

#	Milestone	Weeks	Exit criterion (measured, not felt)	Why here
M5	Harden and launch	34–44	SAML SSO; Stripe billing + plan gating + quotas; load test with a device simulator to 10 000 screens at 60 s heartbeats; external pen test with highs fixed; docs and support runbooks; two weeks of change freeze before launch.	Gating is enforced at the API on entitlements that already exist in the schema; this phase wires UI, billing and operations to them

Why not apps first? They are the most visible feature and the biggest share of the help center, but they sit on top of the asset model and the player. Building them earlier means rebuilding them when the player changes.

Why permissions in M4 and not M0? Roles and the folder permission model need real folders of real resource types and real users to test against. In M0 we lay the schema (`folder.permission`, `membership.role`) and enforce admin-only; the fine-grained model lands when there is something to be fine-grained about.

Why a design partner on screens in M2, not M5? Signage bugs are found on walls, not in staging. Twenty real screens in month 4 is cheaper than two hundred surprised ones in month 10.

5. Estimate

Effort in engineer-weeks (ew), with a confidence per milestone. The base case is the number I would commit to with the mitigations in place; the high case is what it becomes if the named risk lands. Each line is something I can defend individually.

Milestone	Backend	Frontend	Player	Lead	Base ew	High ew	Confidence
M0 Foundations	7	4	1	4	16	18	High
M1 Screen shows a picture	12	9	9	4	34	42	Medium
M2 Playlists, schedules, Android	12	12	16	4	44	58	Medium-low
M3 Layouts, apps, desktop	8	14	5	3	30	36	Medium
M4 Fleet and org	20	14	8	4	46	54	Medium
M5 Harden and launch	14	8	6	6	34	42	Medium-low
Total	73	61	45	25	204	250	

Capacity: 5.5 engineers × 44 weeks = **242 ew** (QA, DevOps and design are not counted as feature capacity). The base case leaves a **16 % buffer**. The high case overruns capacity by 8 ew, about two weeks on paper, but slack is not fungible across tracks: a player-track overrun cannot be absorbed by idle backend weeks. That is why I say **10 months, 11 if M2 or M5 lands on its high case**. Most of the slack sits deliberately in M4 and M5: hardening is where slips from M1 to M3 land, and the launch date is what leadership will hold us to. The M3 and M4 high cases (+6 and +8) are ordinary scope pressure on layouts and the permission UI, not a named risk.

The risks behind the high cases, in order:

1. **Android device variance** (M2, +14 ew). Autostart, WebView versions, storage quotas and display scaling differ by vendor. Help-center evidence: three separate articles on autostart failures across Fire

TV and Android TV, and two on Android display scaling alone. Mitigation: six reference devices in the lab from week 1, a QA owner from M1, the hybrid player so video does not depend on the WebView, and a self-update channel so fixes reach the fleet in a day.

2. **Media pipeline** (M1, +8 ew). Transcoding, resolution ladders, and "why does my video stutter" are a permanent support load. Mitigation: normalise everything to H.264 1080p30 plus a 720p rendition; refuse exotic codecs with a clear error rather than trying to play them.
3. **Launch dependencies outside the team** (M5, +8 ew). Pen-test findings, Stripe edge cases (proration, failed payments, tax), SAML quirks per identity provider. Mitigation: book the pen test in M3 so findings land before M5; limit v1 billing to monthly plans with card payment; test SAML against Okta and Entra only.
4. **Schedule resolution edge cases** (M1/M2, inside the base). Timezones per screen, overlapping rules, default content, DST. Mitigation: one pure engine, property-based tests, the same package on server and player.

If leadership needs a smaller number, Section 7 lists what to remove and what each removal saves.

6. The thing I did not expect, and what it changed

Content access is decided by folders, not by per-item permissions. I assumed a signage portal would let you share an individual screen or playlist with a named user, the way a file is shared in Google Drive. I checked this in the app: the Change Permissions dialog exists on folders in Screens, Files/Assets and Playlists, and a single asset has no such option. Instead, SCIO layers three things: Teams (a workspace whose screens, assets and users are invisible to other teams), Roles (seven defaults plus custom roles that say what a user may do), and then puts the real access control over content on *folders*: a Screens folder set to Admin Only, an Assets folder restricted to the Department A users, a Playlists folder restricted to Department B. The help center's own walkthrough for "let two departments each edit only their zone of a shared screen" is a sequence of folder-permission settings plus a split-screen layout, nothing more. There is no per-asset permission at all; the help center says so outright, and the whole stack is gated to the Pro Plus plan.

Once I saw it, it made sense: the customers who need permissions are multi-location operators, and they think in locations and departments, which map naturally onto folders. A resource-level ACL would be more powerful and much harder to explain to a store manager.

What it changed in the plan

1. **Folders became a first-class entity in the M0 schema**, with `kind` and `permission`, rather than a UI-only grouping added in M4. Every screen, asset, and playlist belongs to exactly one folder from the first migration.
2. **The permission model shrank.** I had budgeted ~6 ew in M4 for a resource-level ACL with policy evaluation. Folder-level inheritance plus a capability set per custom role is ~3 ew. The saved time moved to the Android player, where the help center shows the real pain is.
3. **The public API has to expose folders**, because any integration that creates screens or uploads assets has to say where they go. That was not in my first API sketch.
4. **One deliberate deviation.** OptiSigns applies a parent folder's permission to child folders only at creation; changing the parent later does not propagate. The clone inherits live, with an explicit "break inheritance" on a child. It is the behaviour every admin expects from a file system, and it removes a class of support tickets.

A second surprise, smaller but relevant: the help center documents **two portal UIs side by side** ("1.0" and "Updated"), with every step shown twice. The company is mid-migration of its own frontend. For the clone, that is a strong argument for keeping all logic in the API and the player, and treating the portal as a thin, replaceable client. It confirmed the TypeScript-everywhere, logic-in-the-API decision in Section 2.

7. If the number has to be smaller

Designer, proof-of-play, GraphQL and OAuth apps are already deferred to v1.1 (Section 2). Beyond that, in the order I would cut, with what each cut saves and what it costs the customer:

Cut	Saves	Customer impact
Electron desktop wrapper	3 ew	Windows/mac users use the browser player
Mass provisioning	4 ew	Each screen is paired by hand; fine under 100 screens per customer
Reduce apps from 5 to 2 (YouTube, Website)	4 ew	Framework stays, catalogue grows later
SAML SSO	5 ew	Enterprise-only; design partners use email login
Public REST API + webhooks	6 ew	Blocks enterprise integrations, fine for an SMB launch

Cutting all five takes the base case to **182 ew, about 9 months with the same team**. I would not cut below that: M0–M2 plus split-screen plus teams and folder permissions is the minimum a paying multi-location customer needs.

8. First two weeks, concretely

1. Buy reference devices: Fire TV Stick 4K, two Android sticks of different vendors, an Android tablet, a Chromebox, a Raspberry Pi 4. Hand them to the QA engineer; the device lab is their responsibility from day one.
2. Monorepo (apps/portal, apps/api, apps/player, apps/player-android, packages/schema, packages/schedule-engine), CI, staging on one cloud account, infrastructure as code from the first commit.
3. Schema migration 001: org, team, user, membership, role, folder, screen, asset, playlist, schedule, manifest, audit_log, plan, entitlement, quota. RLS policies on all of them.
4. Spike: pair code flow end to end with a hard-coded image, device token, MQTT over WSS push, web player cache, and the player surviving a pulled network cable. Two engineers, one week. Its result decides whether the manifest design in 3.1 survives.
5. Write the schedule-resolution engine as a pure package with property-based tests before any UI touches it; it ships inside both the API and the player.
6. Sign the first design partner and agree the 20 screens that go live in M2.